

SilentVault: A Password-Based Zero Knowledge File Server

Codreanu Andrei Daniel

Facultatea de Sisteme Informatice si Calculatoare

Academia Tehnica Militara "Ferdinand I"

Bucuresti, Romania

andreidanielcodreanu@gmail.com

Abstract—SilentVault is a secure client-server file storage system that employs a password-based Zero Knowledge Proof authentication protocol based by Schnorr identification. By doing away with reusable password verifiers, the system guarantees that user passwords are never sent or kept on the server. Elliptic curve cryptography is used for authentication, which is carried out over a TLS channel and offers robust security guarantees even in the case of server compromise.

Index Terms—Zero Knowledge Proof, Schnorr Protocol, File Server, Authentication, Cryptography

I. INTRODUCTION

Although password-based authentication is still the most popular method for remote access systems, it has intrinsic flaws such as credential reuse and offline brute-force attacks. Database compromise is a critical security event because traditional solutions based on salted password hashes still expose reusable verifiers to the server.

Unlike traditional password-based schemes that rely on stored verifiers, and unlike PAKE protocols that require complex state management, SilentVault relies on a simple Schnorr-based Zero Knowledge protocol, minimizing server-side secrets while preserving strong authentication guarantees. The server uses algebraic relationships defined over elliptic curve groups to confirm that it knows the password without ever receiving it. The contributions of this work are:

- the design of a password-based Zero Knowledge authentication scheme for file servers using elliptic-curve Schnorr identification;
- an end-to-end client-server architecture integrating ZKP authentication with TLS-secured file operations;
- an open and modular implementation demonstrating the practical applicability of Zero Knowledge Proofs in real-world systems.

II. RELATED WORK

Traditional password-based authentication systems rely on storing salted password hashes derived using key derivation functions such as PBKDF2 or Argon2. While these approaches reduce the feasibility of online attacks, a server compromise enables offline password guessing attacks against stored verifiers, making overall security dependent on password strength and hashing cost parameters.

Password-Authenticated Key Exchange (PAKE) protocols, including SRP, SPAKE2, and OPAQUE, address this limitation by enabling mutual authentication and key establishment without revealing the password to the server. These protocols provide strong security guarantees and resistance to offline attacks, but typically introduce increased protocol complexity and are primarily designed for session key agreement rather than direct identity verification in application-layer services.

Zero Knowledge identification schemes, most notably the Schnorr protocol, allow a prover to demonstrate knowledge of a secret without revealing it. Such schemes have been extensively studied and form the foundation of many modern cryptographic constructions. Their simplicity, well-understood security properties, and compatibility with elliptic curve cryptography make them suitable candidates for password-derived authentication secrets.

Several systems have explored the use of Zero Knowledge Proofs for authentication, particularly in smart card environments and access control mechanisms. However, many of these approaches focus exclusively on the authentication process itself and do not address integration with practical client-server storage architectures.

Secure file storage systems commonly rely on conventional authentication mechanisms combined with encrypted transport channels. In contrast, SilentVault incorporates Zero Knowledge authentication as a first-class component of the storage architecture, demonstrating how Schnorr-based password authentication can be practically integrated into a real-world file server without relying on reusable password verifiers.

III. SYSTEM ARCHITECTURE

SilentVault follows a modular client-server architecture. Communication between the client and server is secured using TLS, while authentication and file operations are handled at the application layer. The client is responsible for user authentication and file operation requests, while the server manages session state, persistent storage, and authentication verification.

Main components include:

- Client module responsible for authentication and file requests
- Server module managing sessions and file storage
- CryptoService implementing the Schnorr-like protocol
- Payload and resolver layers for protocol abstraction

This separation improves maintainability and allows cryptographic components to evolve independently of network logic.

A. Functionalities

In order to provide an example of a ZKP applied to a real-life scenario, SilentVault allows the client to do a couple operations regarding files. He can also manage uploaded files, as follows:

- Uploads files to server.
- Downloads previously uploaded files.
- Checks a list of all the previously uploaded files.
- Demands the server to delete an uploaded file.

The server, besides managing user's files and public data, implements a minimal Command Line Interface, having only the functionality to print a statistic of how many users are currently connected to the server.

Both the client and the server have verbose elements.

B. Responsibilities

1) *Client*: The client is responsible for managing most of the cryptographic functions of the protocol:

- derives the secret x from the password (KDF);
- executes the ZKP Protocol (solving challenge c);
- encrypts the files before sending them to the server;
- decrypts the files after receiving them from the server.

The client is considered a semi-reliable environment. If compromised, only that specific user is compromised, but not the whole system.

2) *Server*: The server acts only as a "worker" for the client. It cannot validate passwords offline or initiate authentications on behalf of the client. The authentication architecture implemented leaves room for future development as a *stateless* authentication method from a cryptographic point of view.

Thus, the server stores only public/already encrypted data. It does not have enough information to execute an impersonation or replay attack.

IV. ZERO KNOWLEDGE AUTHENTICATION PROTOCOL

The implemented ZKP is derived from Schnorr's identification and is implemented over a cryptographic group of elliptic curves.

A. Mathematical Model

The protocol uses an element g to generate a cyclic group G of prime order q . In reality, G is instantiated as a group of elliptic curves, using the NIST standardized P-256 curve.

A user password P is transformed into a secret scalar:

$$x = \text{KDF}(P, \text{salt}) \bmod q \quad (1)$$

The use of a Key Derivation Function is essential against the direct usage of user passwords, which usually have a low entropy from a cryptographic point of view.

The corresponding public value is:

$$Y = g^x \quad (2)$$

B. Registration Flow

Registration (or Enrollment) is done with the help of the password given as input by the user. The steps for enrollment are as follows:

- 1) **Public Key and Salt derivation**: Using the elliptic curve and the user's password, a public key is generated. The salt is generated random.
- 2) **Public Key exchange**: The public key and the salt are sent to the server. It will use the key for future validation of the user, while the user will always generate the public key locally, requesting from the server only the salt used.

C. Authentication Flow

Each authentication session is independent and uses randomly generated values. The steps used in authentication:

- 1) **Commitment phase**: Client selects random $r \in \mathbb{Z}_q$ and computes $R = g^r$. Then, R is sent to the server.
- 2) **Challenge phase**: Server generates a random challenge $c \in \mathbb{Z}_q$ and sends it to the client.
- 3) **Response phase**: Client calculates $s = r + cx \bmod q$. The result s is sent to the server.
- 4) **Verification phase**: Server verifies

$$g^s \stackrel{?}{=} R \cdot Y^c \quad (3)$$

If the equality is satisfied, then authentication is considered a success.

Successful verification proves knowledge of x without revealing it.

D. Security Properties

The protocol satisfies:

- **Completeness**: Honest clients are always accepted. The equation

$$g^s = g^{r+cx} = g^r \cdot (g^x)^c = R \cdot Y^c \quad (4)$$

will always be true.

- Soundness: Attackers cannot generate a valid s for a random challenge c without knowing the secret x . Being able to do that would imply solving the discrete logarithm problem in the elliptic curve group, which is considered to be computationally impossible.
- Zero Knowledge: The protocol never reveals any information about the secret x . For every valid transcript (R, c, s) there is a simulator capable of generating an identical transcript without ever needing to know the secret. Consequently, not even a completely compromised server can extract any useful information about the user's password.

V. COMPARISON WITH CLASSICAL AUTHENTICATION

A. Hash-Based Authentication

Traditional systems store a verifier of the form:

$$H = \text{Hash}(P||\text{salt}) \quad (5)$$

For authentication, the hash is sent to the server, who checks if it matches the stored one. The server is supposed to be a fully reliable participant in the authentication process.

The verifier H is static and can be used by an attacker with an unlimited quota. It allows for password verification without interacting with a legitimate user.

A compromised server is the most critical scenario for traditional systems. It allows offline brute-force attacks, making password safety a time issue. Password security becomes dependent on computational cost alone.

A leak affects all users simultaneously through credential harvesting attacks.

B. Zero Knowledge Approach

Only the public value $Y = g^x$ is stored in SilentVault; this value cannot be used to verify password guesses offline. Authentication is achieved using a protocol in which the client proves their knowledge of the secret without revealing it. It is impossible for a fully compromised server to pose as a client. A compromised database reveals only public data, with no practical value for authentication.

Zero Knowledge authentication implies a higher computational cost and it requires an interactive protocol. Still, the upsides of this approach are the removal of reusable verifiers and protection against offline attacks.

Each authentication is unique and user validation cannot be verified without his participation. A ZK Approach has implications on password re-usage also. If a compromised traditional system poses a threat to multiple platforms because of password re-utilization, SilentVault limits this effect; the passwords cannot be recovered, reducing the risk of a chain reaction.

C. Security Comparison

Property	Classical Authentication	Zero Knowledge
Password exposure	Indirect	None
Reusable Verifier	Yes	No
Offline attack	Yes	No
Replay attack	Possible	Impossible
Server compromise	Critical	Limited
TLS Dependency	Yes	No
Implementation complexity	Low	High
Message Count	Low	High (interactive)
Computational Cost	Low	High

The difference between the two approaches is not incremental, but structural: Classical systems protect the password, ZK eliminates the necessity of protecting that password.

VI. IMPLEMENTATION

A. Modules

The application is implemented using a few modules, some common between the client and the server.

1) *"payload" module*: This module defines the format of the messages sent between the client and the server. It handles serialization and de-serialization, from raw byte strings to concrete classes and vice-versa.

2) *"resolver" module*: This module is specific only to the server. It handles the resolving of different requests received from the client, in a polymorphic manner, according to *Object Oriented Programming* paradigms.

3) *"data" module*: It handles the effective management of stored data on the server, like:

- Reading files.
- Writing files.
- Disk organization.
- Executing a double-encryption on the data received from each server.

B. Languages used

The project is written mainly in C++, but the cryptographic operations are written in *Python* for easier implementation.

VII. CONCLUSION

Zero Knowledge authentication can be successfully incorporated into a workable file server architecture, as SilentVault shows. The system greatly lessens the impact of server compromise and offers a solid basis for secure distributed storage systems by doing away with reusable password verifiers.

Future research will focus on end-to-end encrypted file storage and non-interactive ZK protocols.

REFERENCES

- [1] C. Schnorr, "Efficient Identification and Signatures for Smart Cards," *Advances in Cryptology – CRYPTO*, 1989.
- [2] D. Hankerson, A. Menezes, S. Vanstone, *Guide to Elliptic Curve Cryptography*, Springer, 2004.
- [3] O. Goldreich, *Foundations of Cryptography*, Cambridge University Press.